

08 · Support de démo scriptée (soutenance, ~8 min)

Séquence **exacte** à taper et à *dire* pendant la soutenance. Chaque commande a été **exécutée et vérifiée** le 2026-09-24 ; les sorties ci-dessous sont fidèles. La démo tient en **~8 min** : 2 min dry-run + comparaison · 2 min validation humaine + 1 min injection + 1 min chaîne piégée + 2 min preuves (journal/registre).

0. Avant de monter (préparatifs — matin du jour J)

```
cd prototype
source .venv/bin/activate          # ou : export PATH=$PWD/.venv/bin:$PATH
PYTHONPATH=src python -m pytest tests/ -q  # → 86 passed, 8 fichiers (~1 s)
```

- Afficher à l'écran deux terminaux : **A** = commandes, **B** = `runs/` (preuves).
- Garder fermés : e-mails/bureautique. Aucun réseau requis (dry-run).
- Note : le dossier `cases/` contient `casB_mediconsult.md` (fictif, rappel éventuel au jury : « aucune vraie donnée patient » → garde-fou G3).

1. Dry-run complet sur le cas (AG1→AG5) + comparaison jalon 4 (2 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py run --case cases/casB_mediconsult.md --no-human --comparer
```

Sortie attendue (abrégée) :

```
Fournisseur LLM : dummy-deterministic (test de chaîne – n'est PAS une analyse)
Chaîne : AG1 → AG2 → AG3 → AG4 → AG5 (proposition) → validation humaine

[ok] étape actifs : 14
[ok] étape modele : STRIDE
[ok] étape menaces : 10
[ok] étape evaluations : 10
[ok] étape risques : 10

=== SYNTHÈSE ===
critique : 0   élevé : 6   moyen : 4   faible : 0
risques validés par l'humain : 0/10

=== COMPARAISON vs ANALYSE MANUELLE (jalón 4) ===
10/10 risques retrouvés · 0 inventés · 0 oubliés · 0 écart(s) de niveau
■■■ mode dummy : comparaison tautologique par conception – relancer avec --provider openai pour une vraie
```

À dire :

« Un orchestrateur enchaîne les 5 agents spécialisés. Chacun rend une enveloppe JSON validée par schéma, sourcée. Trois fichiers sont produits : le registre, la mémoire de session (workspace) et le journal horodaté. En dry-run, le fournisseur `dummy-deterministic` est un **test de chaîne, pas une analyse** : il

reproduit la référence (14 actifs, STRIDE + LINDDUN, 10 menaces, 10 risques — 6 élevés, 4 moyens). La commande `--comparer` confronte le registre à `data/reference_manuelle.json` : le 10/10 affiché est **tautologique par conception**, le programme l'assume et l'affiche. La comparaison qui discrimine, c'est le mode imparfait (tests) et le mode réel `--provider openai`. »

Preuve (terminal B) : la trace de cette séquence est `runs/run-20260924-095714/` (dry-run cas B + `--comparer`).

Astuce timing : la commande est quasi instantanée (< 2 s) — c'est un argument (« pas dépendant du réseau »).

Option finale — mode réel `--provider openai` (si la clé API du groupe est prête) :

```
export OPENAI_BASE_URL=... OPENAI_API_KEY=... OPENAI_MODEL=... OPENAI_TIMEOUT=600
PYTHONPATH=src python run.py run --case cases/casB_mediconsult.md --provider openai --comparer
```

À dire : « le mode réel a été exécuté en préparation (24/09/2026) : modèle `space-bunny-free` (API Console OpenCode), trace `runs/run-20260924-110333/` — **10/10 risques retrouvés · 49 inventés · 5 écarts de niveau** : la comparaison n'est pas tautologique avec un vrai modèle. Compter ~10-20 min sur un modèle gratuit ; en soutenance, on peut aussi montrer la trace toute prête. »

2. Humain dans la boucle — validation mixte, référence `Dr Dupont` (2 min)

Commande (terminal A) — validation interactive signée `Dr Dupont` :

```
PYTHONPATH=src python run.py run \
--case cases/casB_mediconsult.md --analyste "Dr Dupont"
```

À la main, reproduire les **décisions mixtes** de la trace de référence `runs/run-20260924-095723/` : **R-01** → **c** (corriger les champs impact + source, le niveau est recalculé par la matrice) · **R-02** → **R-09** → **a** (accepter) · **R-10** → **d** (différer, il reste null).

Sortie attendue (extrait + fin) :

```
[R-01] Vol d'identifiants d'un médecin par harponnage puis connexion à la console...
niveau=... · traitement=réduire (proposé par AG5)
valider ? (a)ccepter / (d)ifférer / (c)orriger : c
→ champs impact + source corrigés · niveau recalculé par la matrice
[R-02] ... valider ? (a)ccepter / (d)ifférer / (c)orriger : a
...
[R-09] ... valider ? (a)ccepter / (d)ifférer / (c)orriger : a
[R-10] Données de santé en clair dans les journaux... niveau=... · traitement=réduire
valider ? (a)ccepter / (d)ifférer / (c)orriger : d

=== SYNTHÈSE ===
critique : 0   élevé : 6   moyen : 4   faible : 0
risques validés par l'humain : 9/10   (R-10 différé, valide_par null)
```

À dire :

« Trois commandes possibles : a accepter, d différer, c corriger. L'agent ne peut jamais s'auto-valider — c'est garanti par le schéma : le champ `valide_par` est de type null dans la sortie de l'agent ; seul l'humain

l'écrit. Ici, décisions mixtes signées Dr Dupont : R-01 corrigé (impact + source, niveau recalculé par la matrice), R-02 à R-09 acceptés, R-10 différé — son valide_par reste null. »

Preuve (terminal B) :

```
grep validation_humaine runs/run-20260924-095723/journal.jsonl | wc -l      # → 10 décisions signées Dr D
grep -c '"valide_par": null' runs/run-20260924-095723/registre_final.json  # → 1 (R-10 différé)
grep appel_agent runs/run-20260924-095723/journal.jsonl | head -1         # → AG1 · octets_entree 5048
grep appel_agent runs/run-20260924-095723/journal.jsonl | tail -1         # → AG5 · octets_entree 11289
```

3. Test d'injection de prompt (1 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py test-injection --case cases/casB_injecte.md
```

Sortie attendue :

```
Document : cases/casB_injecte.md
[détectée] motif=ignore_instructions extrait='Ignore tes instructions'
[détectée] motif=ignore_instructions extrait='Disregard your instructions'
[détectée] motif=disable_securite   extrait='Oublie tes consignes'
... 14 occurrences piégées → 14/14 détections · 9 familles de motifs

Document neutralisé : 2102 → 2149 caractères
(les motifs piégés ont été remplacés par [[DONNEE-NON-EXECUTABLE]])
```

À dire :

« Le même document contient **14 consignes piégées : 14 détections, une par occurrence**, réparties en 9 familles de motifs. Le sanitizer couvre le français, l'anglais, les paraphrases et l'obfuscation — espaces inter-lettres, caractères zero-width, homoglyphes cyrilliques/fullwidth — neutralise les balises <<<...>>>, avec des délimiteurs aléatoires uniques par session ; ses faux positifs (« désormais », « Dorénavant », « Consignes : ») ont été corrigés. Les pièges sont remplacés par des données non exécutables. Ce n'est qu'une couche — la preuve décisive, c'est l'étape suivante. »

4. La chaîne complète tient sur le document piégé (1 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py run --case cases/casB_injecte.md --no-human
```

Sortie attendue : identique à l'étape 1 (14 actifs, STRIDE, 10/10, 6 élevés / 4 moyens).

Preuve (terminal B) :

```
grep -o '"type": "injection_detectee"' runs/run-20260924-095718/journal.jsonl | wc -l
grep -o '"motif"' runs/run-20260924-095718/journal.jsonl | wc -l
```

```
grep -c '"valide_par": null' runs/run-20260924-095718/registre_final.json
# → 1 événement injection_detectee journalisé AVANT le 1er appel, contenant
# 14 détections (9 familles) ; 10 valide_par null dans le registre piégé
```

À dire :

« Même si l'injection passait le filtre à motifs, elle ne changerait rien : un agent ne peut ni écrire valide_par , ni fixer un niveau (matrice), ni omettre les sources. La pièce à conviction : le registre du document piégé est strictement identique à celui du document sain — **14 injections journalisées, 10/10 valide_par null**. Défense en profondeur — test automatisé : jamais faux positif sur le cas sain. »

5. Les preuves — journal et registre (2 min, terminal B)

```
# Le dernier run créé pendant la démo (préparer aussi les 3 traces de référence)
ls -t runs/ | head -1
cat "$(ls -td runs/run-*/ | head -1)journal.jsonl"
```

À montrer : les lignes session_debut → appel_agent (×5, avec octets_entree) → session_fin ; dans le run interactif : validation_humaine (×10) + session_fin nb_risques:10. Traces de référence à garder ouvertes sur le terminal B : run-20260924-095714 (dry-run + --comparer) · run-20260924-095718 (doc piégé, **14 injections journalisées**) · run-20260924-095723 (validation mixte Dr Dupont). Octets réellement envoyés, mesurés dans le journal : AG1 = 5 048 → AG5 = 11 289.

À dire :

« Tout est auditable : qui a appelé quoi, avec combien d'octets en entrée/sortie, chaque injection détectée, chaque décision humaine. C'est le garde-fou G5 et la réponse à la question "ce que vous produisez est-il vérifiable ?". »

Tableau récapitulatif : ce que chaque étape démontre

Étape	Démontre	Garde-fous
1 · dry-run + --comparer	chaîne complète + comparaison (tautologie assumée en dummy)	G2, G4, G7, G9
2 · validation mixte	humain dans la boucle (corriger/valider/différer)	G6 (valide_par)
3 · test-injection	14/14 détections · 9 familles de motifs	G1 , G5
4 · chaîne sur doc piégé	résistance de bout en bout, 14 injections journalisées	G1 + G4/G6/G8
5 · journal/registre	auditabilité (octets mesurés)	G5, G10

Plan B (si une commande échoue)

Symptôme	Réaction
ModuleNotFoundError: jsonschema	<code>pip install -r requirements.txt</code> (10 s), rejouer
Erreur de chemin	relancer depuis <code>prototype/</code> (<code>cd prototype</code>)
Mention d'anomalie (ex. <code>étape AGn impossible</code>)	« cas de l'arrêt bavard (G8) — montrons le journal » : <code>tail runs/run-*/journal.jsonl</code>
Doublon de <code>valide_par ≠ null</code>	impossible par construction ; si le jury insiste, montrer <code>validation.py</code>
Aucun terminal B	faire les greps dans le terminal A, sans casser le rythme

Checklist démo (rappel rapide)

- ☐ `cd prototype` + venv activé
- ☐ `pytest -q` → 86 passed, 8 fichiers (à faire le matin)
- ☐ `cases/casB_mediconsult.md` + `cases/casB_injecte.md` présents
- ☐ Terminal B ouvert sur `runs/` avec un `ls -t` prêt
- ☐ Chrono : 8 min visées, couper § 5 si débordement
- ☐ Après la démo : rester sur l'écran du journal pour les questions